

Performance Comparison of Dijkstra's Algorithm and A* in Hub-and-Spoke Airline Networks

Autenia L. Murray

Georgia Southern University

CSCI 7432 - Algorithms and Data Structures

Instructor: Dr. Yao Xu

May 7, 2026

1. Title

Performance Comparison of Dijkstra's Algorithm and A* in Hub-and-Spoke Airline Networks

2. Team Members

Autenia L. Murray - Georgia Southern University - am16660@georgiasouthern.edu

3. Abstract

This project evaluates Dijkstra's Algorithm and the A* Algorithm on a controlled hub-and-spoke airline network. The network was modeled as a directed, weighted graph using the project's 20-airport historical top-hub sample as nodes and historical OpenFlights route records as directed edges. Edge weights were calculated in kilometers using a deterministic Haversine great-circle distance function. The experiment tested 120 origin-destination pairs, stratified into equal thirds representing short-, medium-, and long-haul routes. Both algorithms were implemented from scratch in Python 3.13 using a binary heap priority queue, and each origin-destination query was timed over 200 repeated runs per algorithm.

The research question guiding the project is: How do Dijkstra's Algorithm and A* compare in shortest-path accuracy, runtime, and node-expansion efficiency on hub-and-spoke airline networks, and how might graph size affect those differences? Results showed 100% shortest-path cost agreement between the algorithms, confirming correctness. A* expanded fewer nodes on 85.0% of tested pairs, with mean expansions of 2.392 nodes compared with 9.325 for Dijkstra. Runtime results were smaller and more sensitive to noise: A* was faster by mean runtime on 56.7% of pairs, with the clearest runtime advantage appearing in long-haul routes. The findings show that A* offers stronger search-efficiency benefits than Dijkstra, but that runtime improvements depend on graph size, heuristic overhead, and network topology. A natural extension is to repeat the experiment on a larger 50- or 100-airport network.

4. Introduction

Shortest-path computation is a central problem in algorithms and data structures because many real systems can be represented as graphs. Airline networks are a useful application because airports can be modeled as vertices, direct flights as edges, and travel distance or cost as an edge weight. In a hub-and-spoke airline network, a small number of hub airports carry a large portion of the connections. This structure can make routing efficient, but it can also concentrate traffic and create vulnerability when a hub is disrupted (Siozos-Rousoulis et al., 2021).

This project compares Dijkstra's Algorithm and A* on an airline shortest-path problem. Dijkstra's Algorithm is a classic shortest-path algorithm for graphs with non-negative edge weights. A* extends the shortest-path idea by adding a heuristic estimate of remaining distance to guide the search toward the target. In this project, the heuristic is the straight-line Haversine distance between the current airport and the destination airport. Because that estimate does not exceed the actual route distance when edge weights are also geographic distances, it is admissible for this simplified model.

The final research question is: How do Dijkstra's Algorithm and A* compare in shortest-path accuracy, runtime, and node-expansion efficiency on hub-and-spoke airline networks, and how

might graph size affect those differences? The project objectives are to implement both algorithms, validate shortest-path correctness, measure runtime, measure node expansions, and interpret how the small 20-airport graph affects the comparison. The report is organized into a literature review, algorithm descriptions, experimental design, results, discussion, conclusion, references, appendices, and contribution statement.

5. Literature Review

Research on air transportation networks supports modeling domestic airline service as a graph with strong hub-and-spoke characteristics. Siozos-Rousoulis et al. (2021) studied the U.S. domestic air transportation network and found that its topology evolved around efficiency and robustness concerns, but also that reliance on important hubs can create structural vulnerability. Pandey and Kafle (2025) similarly describe the U.S. flight network as highly connected and hub-dominant, reinforcing the relevance of shortest-path methods for route planning and disruption analysis.

Dijkstra's Algorithm remains a reliable baseline for non-negative weighted shortest-path problems. Salem et al. (2022) applied Dijkstra's Algorithm to flight scheduling and route selection, showing its usefulness for finding low-cost routes in transportation settings. The main limitation of Dijkstra in single-pair routing is that it does not know where the target is; it expands vertices in increasing distance from the source until the target is reached or all reachable vertices are settled.

A* addresses that limitation by adding heuristic guidance. The original A* formulation by Hart et al. (1968) established that a properly designed heuristic can guide search without losing optimality. Recent transportation research continues to use this idea. Blanco et al. (2022) developed an A*-based method for flight planning and emphasized that admissible and consistent heuristics can improve search performance. Nguyen et al. (2025) applied A* to airway network optimization in the Ho Chi Minh Flight Information Region and reported operational value from reducing route distance and improving network efficiency.

The gap addressed by this project is a controlled, code-based comparison of Dijkstra and A* on a hub-and-spoke airline network using the same graph, same origin-destination pairs, and same distance-based objective. Rather than claiming that one algorithm is always faster, the project separates correctness, runtime, and node expansions. This distinction matters because A* may search fewer vertices while still paying additional overhead to compute the heuristic, especially on very small graphs.

6. Algorithm(s)

6.1 Dijkstra's Algorithm

Dijkstra's Algorithm computes shortest paths from a source vertex to all reachable vertices in a graph with non-negative edge weights. For a single source-target query, the implementation can stop early once the target vertex is removed from the priority queue with its finalized shortest distance.

Pseudocode:

Input: Graph $G = (V, E)$, non-negative weight $w(u, v)$, source s , target t
for each vertex v in V :
 $dist[v] \leftarrow \text{infinity}$; $parent[v] \leftarrow \text{NIL}$
 $dist[s] \leftarrow 0$
insert $(0, s)$ into min-priority queue
while priority queue is not empty:
 remove vertex u with smallest $dist$ value
 if u is stale, skip it
 mark u as expanded
 if $u == t$, stop
 for each neighbor v of u :
 $alt \leftarrow dist[u] + w(u, v)$
 if $alt < dist[v]$:
 $dist[v] \leftarrow alt$
 $parent[v] \leftarrow u$
 insert updated pair into priority queue
return $dist[t]$ and path reconstructed from parent

With a binary heap, Dijkstra's Algorithm runs in $O((|V| + |E|) \log |V|)$ time because each priority queue operation costs logarithmic time. The graph requires $O(|V| + |E|)$ storage, and the distance, parent, expanded, and priority queue structures require $O(|V|)$ additional space. This algorithm was selected as the baseline because it is guaranteed to find optimal paths for non-negative edge weights without requiring a heuristic. This follows the standard priority-queue analysis of shortest-path algorithms (Cormen et al., 2022).

6.2 A* Algorithm

A* is a best-first shortest-path algorithm that orders the priority queue by $f(v) = g(v) + h(v)$. Here, $g(v)$ is the best known distance from the source to v , and $h(v)$ is a heuristic estimate of the remaining distance from v to the target. This project uses Haversine straight-line distance as $h(v)$, which is appropriate for a distance-weighted geographic graph because straight-line distance is a lower-bound estimate of routed travel distance.

Pseudocode:

Input: Graph $G = (V, E)$, source s , target t , heuristic $h(v)$
for each vertex v in V :
 $g[v] \leftarrow \text{infinity}$; $parent[v] \leftarrow \text{NIL}$
 $g[s] \leftarrow 0$
insert $(h(s), 0, s)$ into min-priority queue
while priority queue is not empty:
 remove vertex u with smallest $f = g + h$
 if u is stale, skip it
 mark u as expanded
 if $u == t$, stop
 for each neighbor v of u :
 $tentative \leftarrow g[u] + w(u, v)$
 if $tentative < g[v]$:

```
g[v] <- tentative  
parent[v] <- u  
insert (g[v] + h(v), g[v], v) into priority queue  
return g[t] and path reconstructed from parent
```

In the worst case, A* has the same asymptotic time complexity as Dijkstra, $O((|V| + |E|) \log |V|)$, because an uninformative heuristic can make A* behave like Dijkstra. In practice, an informative admissible heuristic can reduce the number of expanded vertices. Space usage is $O(|V| + |E|)$ for the graph plus $O(|V|)$ for the distance, parent, expanded, and priority queue structures. A* was selected because it extends beyond the baseline shortest-path algorithm and tests whether geographic heuristic guidance improves search efficiency on a hub-and-spoke airline graph. This worst-case framing is consistent with standard graph-search analysis, where heuristic guidance improves practical search behavior without necessarily improving the asymptotic upper bound (Cormen et al., 2022; Hart et al., 1968).

7. Experimentation

7.1 Dataset and Graph Construction

The experiment used the project's 20-airport historical top-hub sample, modeled as a U.S. airline subgraph with hub-and-spoke characteristics. The sample was designed to represent major high-traffic airports in the historical route-data context rather than the exact current FAA top-20 ranking, since airport rankings change over time and the OpenFlights route records are historical (Federal Aviation Administration, 2026; OpenFlights, n.d.). Airport coordinates and route information were derived from OpenFlights airport and route records. OpenFlights route records are directional, meaning that A-to-B and B-to-A routes are separate records when both directions exist. For that reason, the code models the network as a directed weighted graph. The graph contains 20 airport nodes and 293 directed edges, with weights measured in kilometers using the Haversine great-circle distance.

The source data should be interpreted as historical route data rather than a live airline schedule. OpenFlights notes that the third-party route source stopped receiving updates in June 2014, so the graph is best treated as a static historical top-hub experimental network rather than a current operational route map. This limitation does not prevent algorithm comparison because both algorithms run on the same graph, but it does limit real-world claims about present-day airline service.

7.2 Origin-Destination Sampling

The experiment used 120 directed origin-destination pairs out of 380 possible directed pairs among 20 airports. The pairs were divided into equal thirds using direct great-circle distance: 40 short-haul, 40 medium-haul, and 40 long-haul pairs. This design ensures that the comparison includes nearby routes, mid-range routes, and longer routes where heuristic guidance should have more opportunity to reduce the search space.

7.3 Implementation and Validation

Both algorithms were implemented from scratch in Python 3.13 using the standard-library heap queue pattern for the priority queue. Python 3.13 remains supported, and the heap-based priority queue approach remains appropriate for this project. The code was updated to use one

deterministic Haversine function throughout the project so that edge-weight assumptions and A* heuristic estimates are consistent. The code was also updated to log node expansions, relaxations, heap pushes, path cost, hops, and repeated-run timing values.

Correctness was validated by comparing the final shortest-path cost returned by A* against Dijkstra for each origin-destination pair. The updated tests require path-cost equality within a small floating-point tolerance. This is stricter than simply verifying that A* returns a finite route, and it directly checks the claim that the heuristic does not sacrifice optimality.

7.4 Evaluation Metrics

Table 1 lists the main measures used in the experiment and the outcome recorded after rerunning the code.

Metric	What it means	Outcome recorded
Path cost accuracy	Whether Dijkstra and A* returned the same shortest-path distance.	100.0% cost agreement across 120 OD pairs.
Runtime	Average and median wall-clock execution time in milliseconds.	A* faster by mean runtime on 56.7% of pairs; median speedup Dijkstra/A* = 1.173x.
Node expansions	Number of unique vertices removed from the priority queue and processed.	A* fewer expansions on 85.0% of pairs; mean Dijkstra/A* = 9.325 / 2.392.
Hops	Number of directed route edges in the returned path.	Logged for each route as a secondary path-structure measure.
Graph size effect	How the 20-airport graph size affects timing and search behavior.	Interpreted in Discussion; larger-network testing recommended as future work.

Table 1. Metrics used to compare Dijkstra and A*.

8. Results

The final experiment included 120 origin-destination pairs and 200 timing runs per pair for each algorithm. Both algorithms returned the same shortest-path cost for 100.0% of pairs. This confirms that A* preserved Dijkstra-level shortest-path accuracy with the Haversine heuristic.

Metric	Value
Count of OD pairs	120
Runs per OD per algorithm	200
Percent cost agreement	100.0%
Median speedup, Dijkstra time / A* time	1.173
Mean speedup, Dijkstra time / A* time	2.440
Median time saved by A* (ms)	0.0097
Mean time saved by A* (ms)	0.0184
Percent pairs where A* was faster by mean runtime	56.7%
Mean expanded nodes, Dijkstra	9.325

Mean expanded nodes, A*	2.392
Percent pairs where A* expanded fewer nodes	85.0%

Table 2. Overall performance summary for 120 origin-destination pairs.

Haul category	Pairs	Mean Dijkstra ms	Mean A* ms	Median Dijkstra ms	Median A* ms	Mean Dijkstra expansions	Mean A* expansions	A* fewer expansions
Short	40	0.1163	0.1234	0.0289	0.0453	3.675	2.375	55.0%
Medium	40	0.1489	0.1636	0.0470	0.0452	8.525	2.450	100.0%
Long	40	0.1409	0.0640	0.0664	0.0446	15.775	2.350	100.0%

Table 3. Results by equal-thirds haul category.

The runtime results are mixed but informative. Overall, A* was faster by mean runtime on 56.7% of pairs, and the median speedup was 1.173x. However, the absolute timing values are extremely small because the graph has only 20 nodes. This means that runtime measurements are close to the noise floor for Python timing. The clearest runtime advantage appeared in the long-haul category, where A* had both lower mean runtime and lower median runtime than Dijkstra. In short-haul routes, Dijkstra was faster because the target is often reached quickly and A* still pays heuristic-computation overhead. The figures in this section were generated with Matplotlib to visualize runtime and node-expansion comparisons (Hunter, 2007).

The node-expansion results are stronger and more stable. Dijkstra expanded 9.325 nodes on average, while A* expanded 2.392 nodes on average. A* expanded fewer nodes on 85.0% of all tested pairs. In medium- and long-haul categories, A* expanded fewer nodes on 100% of pairs. This supports the conclusion that A* searches more selectively even when wall-clock runtime differences are small.

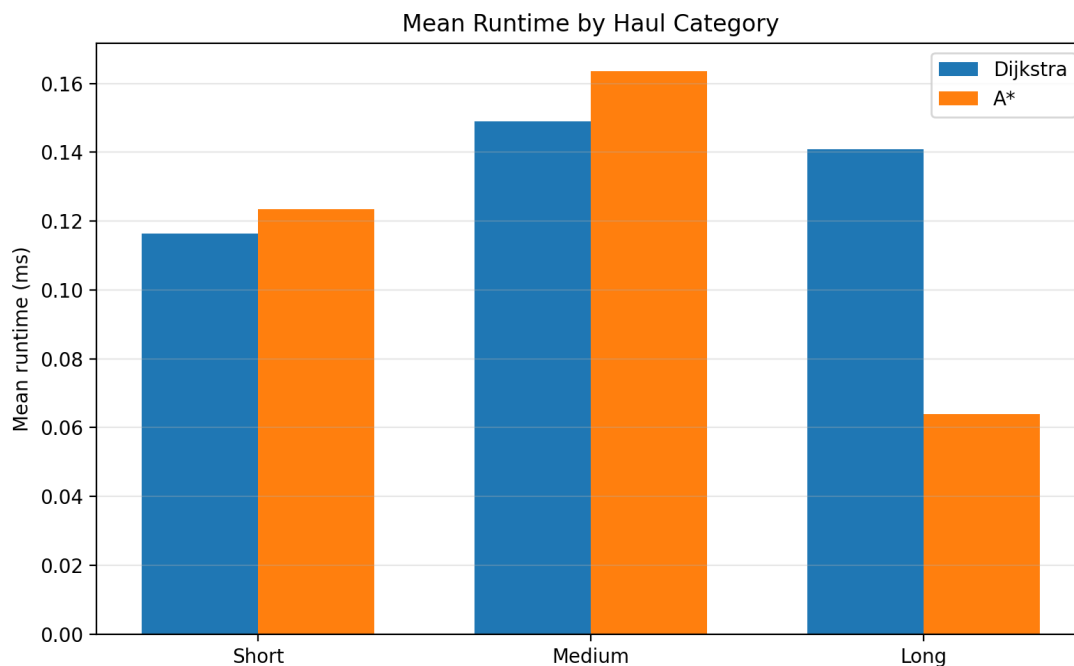


Figure 1. Mean runtime by haul category.

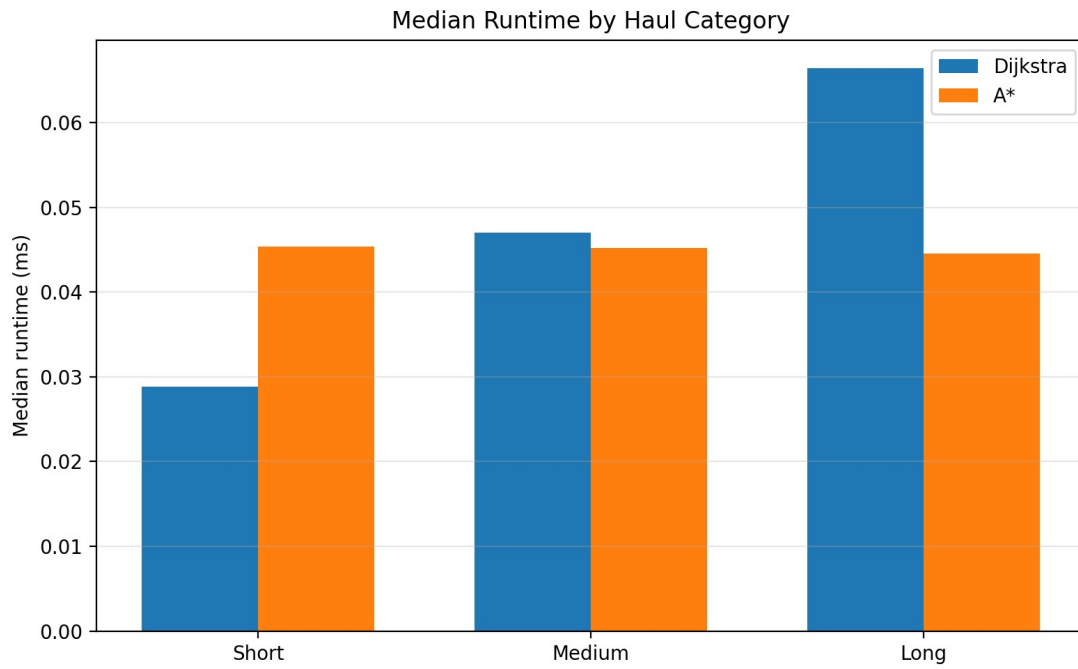


Figure 2. Median runtime by haul category.

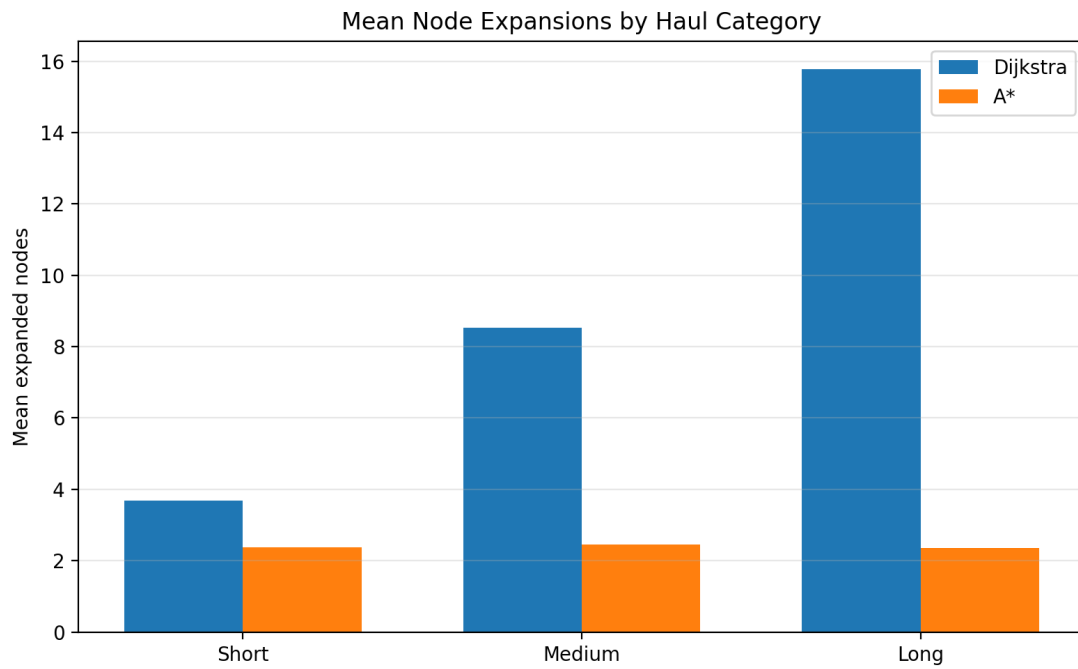


Figure 3. Mean node expansions by haul category.

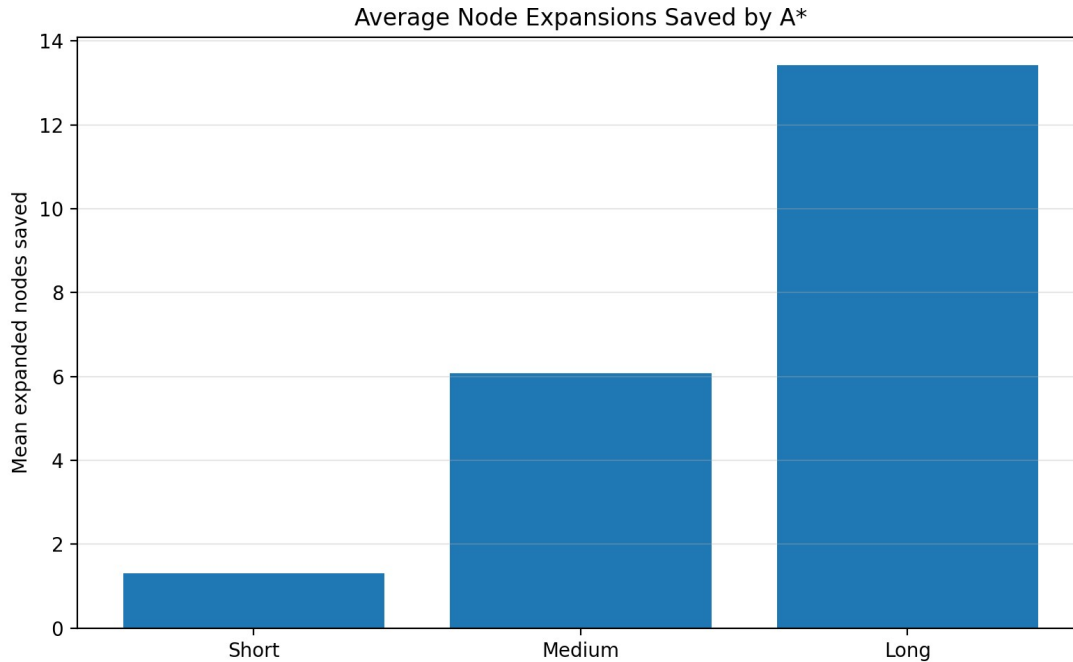


Figure 4. Average node expansions saved by A*.

9. Discussion

The main finding is that A* preserved shortest-path correctness while reducing search effort. The 100% cost agreement shows that the Haversine heuristic did not change the answer produced by Dijkstra. This matches the theoretical expectation for A* with an admissible geographic heuristic and supports prior work showing the value of heuristic search in transportation networks (Blanco et al., 2022; Nguyen et al., 2025).

The runtime results require careful interpretation. A* did not simply dominate Dijkstra in every timing measure. On short routes, Dijkstra often ran faster because the graph was tiny and the target could be reached after only a few expansions. A* still had to compute heuristic values, so the extra arithmetic could offset the search reduction. This is an important algorithmic lesson: theoretical search efficiency and wall-clock runtime are related, but they are not identical. Implementation overhead, graph size, route length, and hardware timing noise all influence measured runtime.

The strongest evidence for A* appears in node expansions. A* reduced mean expansions from 9.325 to 2.392 nodes, and the reduction increased with route distance. For medium- and long-haul routes, A* expanded fewer nodes in every tested case. This means A* was better at avoiding unnecessary exploration, especially when the source and destination were farther apart. For larger airline graphs, this reduction would be expected to matter more because each avoided expansion could prevent many additional edge relaxations.

The small graph size is the main limitation of the present experiment. A 20-node graph is appropriate for a controlled class project and makes the algorithm behavior easy to inspect, but it is too small to fully demonstrate scalability. A next step would be to repeat the same experimental design on the top 50 or top 100 U.S. airports and compare whether A*'s lower

node-expansion count produces clearer runtime advantages as the graph grows. This would directly test the graph-size effect included in the research question.

Another limitation is that the graph is static and distance-based. Real airline routing depends on schedules, fares, aircraft availability, delays, airspace restrictions, and time-dependent connections. Future work could extend the graph to include travel time, connection time, or multi-objective weights. Additional shortest-path methods such as bidirectional Dijkstra, bidirectional A*, or ALT with landmarks could also be compared.

10. Conclusion

This project compared Dijkstra's Algorithm and A* on a directed, weighted hub-and-spoke airline graph. Both algorithms were implemented from scratch in Python and tested on the same 120 origin-destination pairs. The algorithms produced 100% shortest-path cost agreement, confirming that A* preserved optimality under the Haversine heuristic.

The results show that A* was more search-efficient than Dijkstra. It expanded fewer nodes on 85.0% of all tested pairs and reduced mean expansions from 9.325 to 2.392 nodes. Runtime improvements were present but smaller and more sensitive to graph size and overhead. A* was faster by mean runtime on 56.7% of pairs, with the clearest timing benefit in long-haul routes. Therefore, the most accurate conclusion is not that A* is always faster, but that A* searches more selectively and may produce stronger runtime benefits as graph size increases.

The next step is to rerun the same benchmark on a larger network, such as 50 or 100 airports. That extension would test whether A*'s lower node-expansion count scales into a stronger runtime advantage. Overall, the project demonstrates how algorithm selection depends on correctness, theoretical complexity, implementation details, and network structure.

11. References

- Blanco, M., Borndorfer, R., & Maristany de las Casas, P. (2022). An A* algorithm for flight planning based on idealized vertical profiles. In 22nd Symposium on Algorithmic Approaches for Transportation Modelling, Optimization, and Systems (ATMOS 2022), 1:1-1:15. <https://doi.org/10.4230/OASICS.ATMOS.2022.1>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.
- Federal Aviation Administration. (2026). Passenger boarding (enplanement) and all-cargo data for U.S. airports. https://www.faa.gov/airports/planning_capacity/passenger_allcargo_stats/passenger
- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100-107. <https://doi.org/10.1109/TSSC.1968.300136>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95. <https://doi.org/10.1109/MCSE.2007.55>
- Nguyen, N. H. Q., Vu, T. H. A., Le, H. M., Nguyen, M. P., Le, T. T. T., Nguyen, T. T. T., & Nguyen, N. N. Y. (2025). Optimizing airway network efficiency with the A-star

- algorithm: A case study of Ho Chi Minh FIR. *Aviation*, 29(2), 104-117.
<https://doi.org/10.3846/aviation.2025.23635>
- OpenFlights. (n.d.). OpenFlights: Airport and airline data. Retrieved May 7, 2026, from
<https://openflights.org/data.php>
- Pandey, S., & Kafle, S. (2025). Analysis and resilience of the U.S. flight network. arXiv preprint arXiv:2505.11559.
- Python Software Foundation. (2026). heapq - Heap queue algorithm. Python 3.13 documentation. <https://docs.python.org/3.13/library/heapq.html>
- Python Software Foundation. (2026). Status of Python versions. Python Developer's Guide. <https://devguide.python.org/versions/>
- Salem, I. E., Mijwil, M. M., Abdulqader, A. W., & Ismaeel, M. M. (2022). Flight-schedule using Dijkstra's algorithm with comparison of routes findings. *International Journal of Electrical and Computer Engineering*, 12(2), 1675-1682.
<https://doi.org/10.11591/ijece.v12i2.pp1675-1682>
- Siozos-Rousoulis, L., Robert, D., & Verbeke, W. (2021). A study of the U.S. domestic air transportation network: Temporal evolution of network topology and robustness from 2001 to 2016. *Journal of Transportation Security*, 14(1-2), 55-78.
<https://doi.org/10.1007/s12198-020-00227-x>

12. Appendices

Appendix A: Final Code and Result Files

The accompanying source-code package includes the updated Python implementation, tests, final result CSV files, and generated figure images. Important files include `src/algorithms/dijkstra.py`, `src/algorithms/astar.py`, `src/utils/haversine.py`, `src/graph/graph_builder.py`, `tests/test_real_graph.py`, `tools/run_final_experiment.py`, `results/final_200/final_results_merged.csv`, `results/final_200/final_results_summary.csv`, and `results/final_200/figures/`. The `graph_builder.py` script builds the route-network visualization used as a supporting visual aid; the official reproducible benchmark remains `tools/run_final_experiment.py`. To reproduce the final experiment from the code-package root, run: `python tools/run_final_experiment.py --runs 200 --outdir results/final_200`.

Appendix B: Contribution Statement

All work completed independently by Autenia L. Murray.